Functional Dependencies

Armstrong’s Axioms in Functional Dependency in DBMS

Finding Attribute Closure and Candidate Keys using Functional Dependencies

Functional Dependency

- **Functional Dependency (FD)** is a constraint that determines the relation of one attribute to another attribute in a Database Management System (DBMS). A functional dependency is denoted by an arrow “→”. The functional dependency of X on Y is represented by X → Y. Let’s understand Functional Dependency in DBMS with example.

Functional Dependency

Employee number	Employee Name	Salary	City
1	Dana	50000	San Francisco
2	Francis	38000	London
3	Andrew	25000	Tokyo

Example:
In this example, if we know the value of Employee number, we can obtain Employee Name, city, salary, etc.

By this, we can say that the city, Employee Name, and salary are functionally depended on Employee number.

Functional Dependency

What does functionally dependent mean?

A function dependency A → B means for all instances of a particular value of A, there is the same value of B.

For example in the below table A → B is true, but B → A is not true as there are different values of A for B = 3.

A	B
1	3
2	3
4	0
1	3
4	0

Types of Functional Dependency

1. Trivial Functional Dependency

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant.

i.e. If **X → Y** and **Y is the subset of X**, then it is called trivial functional dependency

For example,

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **{roll_no, name} → name** is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}**

Similarly, **roll_no → roll_no** is also an example of trivial functional dependency.

Functional Dependency

2. Non-trivial Functional Dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.
i.e. If **X → Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

For example,

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, **roll_no → name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**
Similarly, **{roll_no, name} → age** is also a non-trivial functional dependency, since **age** is **not a subset of {roll_no, name}**

Functional Dependency

3. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant.

i.e. If **a** → **b** & **b** → **c**, then according to axiom of transitivity, **a** → **c**. This is a **transitive functional dependency**

For example,

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, **enrol_no** → **dept** and **dept** → **building_no**,

Hence, according to the axiom of transitivity, **enrol_no** → **building_no** is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Functional Dependency

- Semi Non Trivial Functional Dependencies
- X → Y is called ***semi non-trivial*** when X intersect Y is not NULL.
- Examples:
- AB → BC,
- AD → DC
- For e.g (R.NO,Name)->(Name,Marks)

Note:

“Trivial Dependency are always valid ,But Non-Trivial Dependency varies relation to relation.”

Armstrong’s Axioms in Functional Dependency in DBMS

- The term Armstrong axioms refer to the sound and complete set of inference rules or axioms, introduced by William W. Armstrong, that is used to test the logical implication of **functional dependencies**.
- Consider example table

RNO	Name	Marks	Dep	Course
1	a	78	CS	C1
2	b	60	EE	C1
3	a	78	CS	C2
4	b	60	EE	C3
5	c	80	IT	C2

Armstrong’s Axioms in Functional Dependency in DBMS

- Axiom of reflexivity –

1. **Axiom of reflexivity -**

If A is a set of attributes and B is subset of A, then A holds B. If $B \subseteq A$ then $A \rightarrow B$ This property is trivial property.

Armstrong’s Axioms in Functional Dependency in DBMS

- Axiom of transitivity –

Same as the transitive rule in algebra, if $A \rightarrow B$ holds and $B \rightarrow C$ holds, then $A \rightarrow C$ also holds.
$A \rightarrow B$ is called as A functionally that determines B. If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

- If Name->Marks and Marks ->Dep. Then Name -> Dep

Armstrong’s Axioms in Functional Dependency in DBMS(Secondary)

- Axiom of UNION –

1. **Union –**
   If $A \rightarrow B$ holds and $A \rightarrow C$ holds, then $A \rightarrow BC$ holds. If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$

For R.N->NAME,R.N->Marks,then we can determine R.N->(Marks,Name)

Armstrong’s Axioms in Functional Dependency in DBMS(Secondary)

- Axiom of Decomposition –

Decomposition –
If $A \rightarrow BC$ holds then $A \rightarrow B$ and $A \rightarrow C$ hold. If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$

For e.g if Name,Marks->Dept,Course then Name,Marks->Dept and Name,Marks->Course

Armstrong’s Axioms in Functional Dependency in DBMS(Secondary)

- Axiom of Composition –

Composition –
If $A \rightarrow B$ and $X \rightarrow Y$ holds, then $AX \rightarrow BY$ holds.

Armstrong’s Axioms in Functional Dependency in DBMS(Secondary)

- Axiom of Pseudo Transitivity –

If $A \rightarrow B$ holds and $BC \rightarrow D$ holds, then $AC \rightarrow D$ holds. If $X \rightarrow Y$ and $YZ \rightarrow W$ then $XZ \rightarrow W$.

Finding Attribute Closure and Candidate,Super Keys using Functional Dependencies

- Attribute Closure:

  "Closure of an **attribute x** is the set of all attributes that are functional dependencies on X with respect to F. It is denoted by X^+ which means what X can determine."

- Example

R(A,B,C,D,E) AND F: A->B,B->C, C->D, A->E. Find the closure of F

Solution

A^+= {A,B,C,D,E}

B^+= {B,C,D}

C^+= {C,D}

F^+= {A->A, A->B, A->C, A->D, A->E, B->B, B->C, B->D, C->C, C->D}

Finding Attribute Closure and Candidate,Super Keys using Functional Dependencies

In EMPLOYEE relation given in Table 1,

- FD **E-ID->E-NAME** holds because for each E-ID, there is a unique value of E-NAME.
- FD **E-ID->E-CITY** and **E-CITY->E-STATE** also holds.
- FD E-NAME->E-ID **does not hold** because E-NAME 'John' is not uniquely determining E-ID. There are 2 E-IDs corresponding to John (E001 and E003).

EMPLOYEE

E-ID	E-NAME	E-CITY	E-STATE
E001	John	Delhi	Delhi
E002	Mary	Delhi	Delhi
E003	John	Noida	U.P.

Table 1

Finding Attribute Closure and Candidate,Super Keys using Functional Dependencies

Given R(E-ID, E-NAME, E-CITY, E-STATE)

```
FDs = { E-ID->E-NAME, E-ID->E-CITY, E-ID->E-STATE, E-CITY->E-STATE }
```

The attribute closure of E-ID can be calculated as:

1. Add E-ID to the set {E-ID}
2. Add Attributes which can be derived from any attribute of set. In this case, E-NAME and E-CITY, E-STATE can be derived from E-ID. So these are also a part of closure.
3. As there is one other attribute remaining in relation to be derived from E-ID. So result is:

$$(E\text{-}ID)^+ = \{E\text{-}ID, E\text{-}NAME, E\text{-}CITY, E\text{-}STATE\}$$

Similarly,

$$(E\text{-}NAME)^+ = \{E\text{-}NAME\}$$
$$(E\text{-}CITY)^+ = \{E\text{-}CITY, E\_STATE\}$$

Finding Attribute Closure and Candidate,Super Keys using Functional Dependencies

Candidate Key

Candidate Key is **minimal set of attributes** of a relation which can be used to identify a tuple uniquely. For Example, each tuple of EMPLOYEE relation given in Table 1 can be uniquely identified by **E-ID** and it is minimal as well. So it will be Candidate key of the relation.

A candidate key may or may not be a primary key.

Super Key

Super Key is **set of attributes** of a relation which can be used to identify a tuple uniquely.For Example, each tuple of EMPLOYEE relation given in Table 1 can be uniquely identified by **E-ID or (E-ID, E-NAME) or (E-ID, E-CITY) or (E-ID, E-STATE) or (E_ID, E-NAME, E-STATE)** etc. So all of these are super keys of EMPLOYEE relation.

> **Note:** A candidate key is always a super key but vice versa is not true.

Finding Attribute Closure and Candidate,Super Keys using Functional Dependencies

Q. Finding Candidate Keys and Super Keys of a Relation using FD set The **set of attributes** whose attribute closure is set of all attributes of relation is called super key of relation. For Example, the EMPLOYEE relation shown in Table 1 has following FD set. **{E-ID->E-NAME, E-ID->E-CITY, E-ID->E-STATE, E-CITY->E-STATE}** Let us calculate attribute closure of different set of attributes:

```
(E-ID)+ = {E-ID, E-NAME,E-CITY,E-STATE}
(E-ID,E-NAME)+ = {E-ID, E-NAME,E-CITY,E-STATE}
(E-ID,E-CITY)+ = {E-ID, E-NAME,E-CITY,E-STATE}
(E-ID,E-STATE)+ = {E-ID, E-NAME,E-CITY,E-STATE}
(E-ID,E-CITY,E-STATE)+ = {E-ID, E-NAME,E-CITY,E-STATE}
(E-NAME)+ = {E-NAME}
(E-CITY)+ = {E-CITY,E-STATE}
```

As $(E\text{-}ID)^+$, $(E\text{-}ID, E\text{-}NAME)^+$, $(E\text{-}ID, E\text{-}CITY)^+$, $(E\text{-}ID, E\text{-}STATE)^+$, $(E\text{-}ID, E\text{-}CITY, E\text{-}STATE)^+$ give set of all attributes of relation EMPLOYEE. So all of these are super keys of relation.

The **minimal set of attributes** whose attribute closure is set of all attributes of relation is called candidate key of relation. As shown above, $(E\text{-}ID)^+$ is set of all attributes of relation and it is minimal. So E-ID will be candidate key. On the other hand $(E\text{-}ID, E\text{-}NAME)^+$ also is set of all attributes but it is not minimal because its subset $(E\text{-}ID)^+$ is equal to set of all attributes. So (E-ID, E-NAME) is not a candidate key.